

CADIO Project Overview

1. The Problem

Making a real 3D model today is hard, and the hardness is in the wrong place.

Traditional CAD tools (SolidWorks, Fusion 360, Blender) are powerful but have a steep learning curve. You need to know sketches, constraints, extrudes, lofts, booleans, and dozens of menus before you can make even a simple box with a hole in it. Most people with a good idea for a physical object simply can't build it themselves. They have the *idea* but not the *skill*.

AI "text-to-3D" generators went the other way. You type a prompt and get a mesh, but that mesh is a dead end:

- It's a blob of triangles you **can't edit meaningfully**. There's no "make the wall 2mm thicker" knob.
- It's **not dimensionally accurate**, so it's useless for anything you actually want to manufacture or 3D-print.
- It **can't be exported** in the formats real workflows need (STEP for manufacturing, precise STL for printing).
- The AI is a **slot machine**: you get one shot, and if it's wrong, you re-roll and hope.

So the market has two bad options: tools that are precise but hard to use, and tools that are easy to use but produce throwaway results.

What's actually needed

A tool where you **describe** what you want in plain language, **collaborate** with an intelligent agent that asks the right questions, and get back a model that is:

1. **Editable**: every dimension is a knob you can turn.
2. **Precise**: real millimeters, real geometry, validated for correctness.
3. **Production-ready**: exports to the formats manufacturing and printing use.
4. **Never lost**: your conversations, versions, and files all persist.

2. What CADIO Is

CADIO is an **AI-native 3D creation platform**. You talk to a design agent the way you'd talk to a junior engineer, and together you produce a fully editable, production-ready 3D model.

The core idea:

You describe the idea; an AI agent writes the CAD program; a precise geometry engine builds it; and you fine-tune the result with sliders, all without ever losing your work.

The key insight that makes CADIO different from other AI-3D tools: **the AI doesn't generate a mesh directly. It generates a small parametric program.** That program (**PARAMS** + a **build()** function) is real CAD code. Because the output is a *program*, not a *blob*:

- Every requirement number becomes a **tweakable parameter** (a slider).
- The geometry is **exact** (built by a real CAD kernel, not approximated).
- Changing a slider **re-runs the program locally in milliseconds**, with no AI call, no cost, and no waiting.
- You can **read and edit the code** yourself if you want.

The mental model: a generic shell + pluggable engines

CADIO is designed as a **generic shell** wrapped around **pluggable geometry engines**.

- The **shell** is everything that's the same no matter what kind of model you make: the conversation, the intent, the version history, the 3D viewer, the validation gate, the export system, the persistence.
- An **engine** is the thing that actually turns a program into geometry. Today there is one engine (the *precision* engine, based on build123d). Tomorrow there will be more (Blender for organic and stylized models, generative engines later).

This split is the central architectural bet: **breadth comes from adding engines, depth comes from perfecting each one**, but the shell never changes. The shell talks to every engine through one small contract, so it never needs to know how any particular engine works.

3. Features

Here's what CADIO can do today, grouped by what it means for the user.

The conversation (the agent)

- **Describe-to-build.** Tell the agent what you want in plain language; it builds it.
- **Asks the right questions.** Instead of guessing, the agent gathers requirements first (batched Q&A with sensible defaults), like a real engineer taking a brief.
- **The agent has eyes.** After each build, the agent *renders* the model from four angles and *looks at its own work* using a vision model. It catches shape problems and fixes them **before** showing you the result.
- **Reference images.** Attach photos of what you want (paperclip, drag-and-drop, or a reference library). The agent describes what it sees and confirms the object before building.
- **Reference geometry.** Upload existing STEP/STL files; the agent measures them (bounding box, volume, bore diameters) so "make a lid that fits this" works with real numbers.
- **Interruptible.** Hit stop mid-thought; the agent cancels cleanly.
- **Provider-agnostic.** Bring your own API key for Claude, OpenAI, Gemini, or xAI/Grok, and pick the model from a live list in the UI.

The model (editing and precision)

- **Live parameter sliders.** Every requirement number becomes a slider or input. Drag it and the model rebuilds in about 20ms, **with no AI call and no cost**.
- **Part-aware selection.** Click a face or part in the 3D viewer and CADIO tells you which part it is and which parameters control it, with named parts, selection chips, and per-part parameter panels.

- **Parameter highlighting.** Click a parameter and the exact faces it controls glow in the viewer, so you can see what each knob does.
- **Raw code editing.** Power users can open the actual build123d program, edit it, and re-run.
- **Validation gate.** Every model is checked automatically for watertightness, correct winding, positive volume, and a cross-check that the exported mesh matches the exact CAD geometry (which catches silent export bugs).

The workspace (persistence and versions)

- **Nothing is ever lost.** Everything lives in named projects: conversations, every model version, reference images, exports. Close the laptop, come back tomorrow, continue the same conversation with full memory.
- **Refresh-safe builds.** A build keeps running even if you refresh the page or briefly disconnect; when you reconnect, you re-adopt the in-flight build.
- **Version history.** Every build is a numbered version (v1, v2, v3, and so on) tagged by origin (agent-built vs. manually saved). Click any version to reload it.
- **The agent edits what's on screen.** If you load an older version, the agent knows that's the model you're looking at and edits *that* one.
- **Exports on demand.** Download STL (3D printing), STEP (manufacturing/CAD), or GLB (web/visualization) any time.

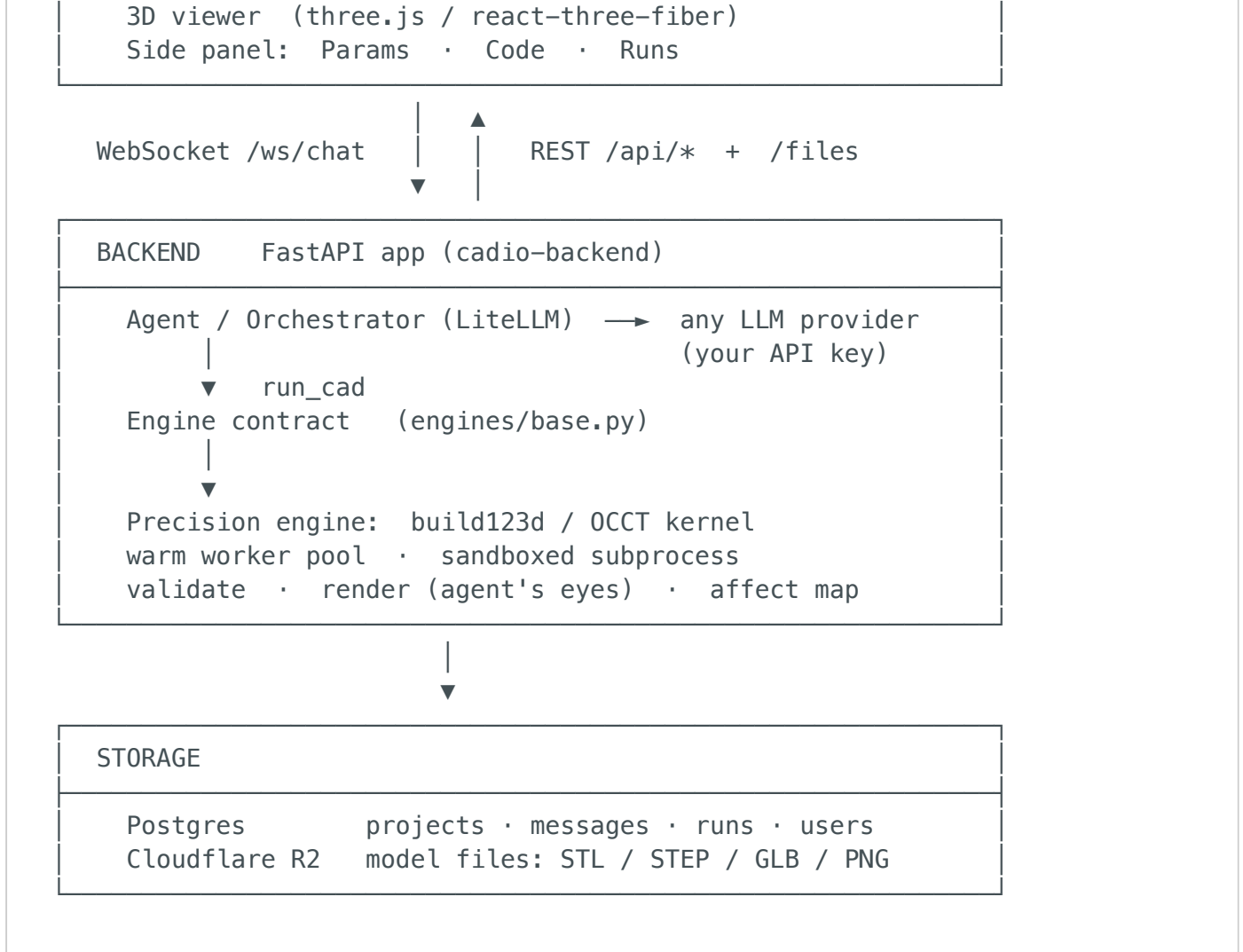
Access

- **Web workspace**, the primary surface: chat, a 3D viewer, and a tabbed side panel.
- **Google sign-in** and per-user projects (in the deployed version).

4. Architecture

CADIO is two applications that talk over HTTP and a WebSocket, backed by a store for data and files.





How a single request flows

1. **You type a message** in the browser. It goes over the WebSocket to the backend, along with your model choice and (for this session only) your API key.
2. **The orchestrator** (`agent/orchestrator.py`) runs an AI tool-use loop. The agent can call two tools: `ask_user` (to ask clarifying questions) and `run_cad` (to build a model).
3. When the agent calls `run_cad`, it hands a Python program (`PARAMS + build()`) to the **engine**.
4. **The precision engine** runs that program in a **sandboxed subprocess** from a **warm worker pool**: a resident Python process with the heavy CAD kernel already loaded, so a build takes about 20ms instead of about 6 seconds cold.
5. The engine **exports** the geometry, **validates** it (watertight, winding, volume, mesh-vs-CAD bbox), **renders** four views for the agent's eyes, and returns **measured facts** (real bounding box, volume).
6. The agent **looks at the renders**, checks the measured facts against your requirements, fixes anything wrong, and then **presents the result**.

7. Everything (the message, the run, the artifacts) is **persisted** to the store, and the browser updates the viewer and the sliders live.

Key architectural decisions

- **The engine contract is sacred.** The shell only ever talks to `Engine.execute(code, params, run_dir)`. It never imports engine internals. This is what lets new engines (Blender, generative) slot in without touching the agent, API, or UI.
- **Sliders never call the AI.** Re-running a program with new parameter values is pure local compute. This makes tweaking instant and free, and is why every requirement is modeled as a parameter.
- **The session outlives the socket.** An agent turn runs in a session-owned background thread, not tied to your WebSocket. That's why a page refresh mid-build doesn't lose the build; you just re-attach to it.
- **One serializer for two audiences.** Conversation history is serialized once and used both to rebuild the agent's memory (LLM history) and to draw the chat scrollback (UI history). This keeps them from ever drifting apart, so the agent genuinely remembers across restarts.
- **Store the source, regenerate the rest.** The only irreplaceable data is the *source* (the program + parameters + chat), just a few kilobytes of text. The heavy files (STEP, full-resolution STL) are deterministically rebuildable, so they're generated **on demand at export time** rather than stored. A small decimated mesh is cached for the viewer.
- **Compute is the scaling limit, not storage.** Because users bring their own LLM keys, the expensive AI inference is offloaded. The server's scarce resource is **CPU for geometry**, since each build is a CPU-bound CAD subprocess. That is why the bottleneck is how many models are built at once, not how much data is stored, and why scaling out means adding compute, not just storage.

5. Tech Stack and Why Each Piece Matters

Backend

Technology	What it does	Why we need it
Python 3.12	Language for the whole backend	The CAD kernel (OpenCASCADE/OCP) only ships as Python wheels, and Python 3.12 is the version those wheels support (3.14 is too new; 3.10 to 3.12 is the supported window).
build123d	The CAD modeling library the agent writes programs against	Gives a clean, code-first way to describe precise solids (boxes, revolves, lofts, sweeps, fillets). It's the "language" the AI generates.
OpenCASCADE / OCCT (via OCP)	The industrial geometry kernel underneath build123d	This is the same class of kernel professional CAD tools use. It's what makes the geometry <i>exact</i> and <i>manufacturable</i> , and what produces true B-rep STEP files.
trimesh + manifold3d + fast-simplification	Mesh handling, boolean robustness, decimation	Validates meshes (watertight, winding), and shrinks big meshes down to a light "display mesh" for the viewer without hurting the exact model.
shapely + rtree + networkx	2D geometry, spatial indexing, graph ops	Support the validation and the "which parameter moves which faces" (affect map) computations, such as nearest-surface diffing and cross-section slicing.
matplotlib	Headless rendering of the model to PNGs	This is the agent's eyes : it renders four views so a vision model can critique the shape. It also doubles as the project thumbnail.
FastAPI + Uvicorn	The web server, REST endpoints, and WebSocket	Async-native (needed for the long-lived chat WebSocket), fast, and gives typed request/response handling with almost no boilerplate.
LiteLLM	One unified interface to every LLM provider	Lets the <i>same</i> agent loop run on Claude, OpenAI, Gemini, or xAI without provider-specific code. Users bring any key; the agent code doesn't change.

Technology	What it does	Why we need it
Pydantic	Data validation and shapes	Keeps the manifest, params, and API payloads well-typed and safe.
Postgres	The metadata store: users, projects, conversations, runs, and assets	This is the text data that must never be lost. Postgres handles many users at once and lets more than one server share the same data (run as a managed Supabase database in deployment).
boto3 / Cloudflare R2	Object storage (S3-compatible) for artifacts	Model files live in durable, CDN-able storage instead of on one machine's disk.
Authlib + itsdangerous	Google OAuth sign-in + signed session cookies	Real per-user accounts and sessions that survive server restarts.
orjson	Fast JSON	Required by LiteLLM on the completion path; also speeds up serialization.

Frontend (**cadio-frontend**)

Technology	What it does	Why we need it
React 19 + TypeScript	The UI framework and type safety	A live, stateful workspace (chat + viewer + panels updating together) is exactly what React is for; TypeScript keeps the API contract honest.
Vite	Dev server + build tool	Instant hot-reload in dev (with a proxy to the backend) and an optimized static bundle in prod. Chosen over Next.js because there's no SSR or SEO need: it's a canvas and WebSocket app.
three.js + @react-three/fiber + drei	The 3D viewer	Renders the model in the browser, handles camera framing, and, most importantly, lets us recolor individual triangles for part selection and parameter highlighting. fiber

Technology	What it does	Why we need it
		makes three.js declarative in React, and dreii adds camera and bounds helpers.
react-router-dom	Routing (/ home, /p/<id> workspace)	Named projects need real URLs you can share and reload into.
react-resizable-panels	The draggable three-pane layout	Users resize the chat, viewer, and side panel to taste.
react-markdown	Renders the agent's replies	Agent messages come back as markdown.
Playwright	Visual/screenshot testing	Catches breakage that API tests can't, such as a route accidentally shadowing the app bundle and blanking the page.

Infrastructure

- **Docker:** the backend and frontend each ship as a container.
- **Bifrost:** the deployment platform (dev environment). It runs the frontend (an nginx single-page app) and the backend as two independent services wired together by environment variables, so a failing backend deploy can't take the frontend down.

6. Why the "program, not a mesh" bet matters

It's worth restating the single decision everything else hangs on, because it's what makes CADIO fundamentally different from every "AI makes 3D" tool:

Other tools ask the AI to output the final geometry. CADIO asks the AI to output a small program that *generates* the geometry.

That one difference buys us:

- **Editability:** the program has named parameters, so every number is a knob.

- **Precision:** a real CAD kernel executes the program, so the result is exact.
 - **Speed and zero cost on tweaks:** turning a knob re-runs the program locally, with no AI round-trip.
 - **Correctness:** the program can *assert* its requirements, and we can *measure* the built geometry and check it.
 - **Transparency:** you can read and edit the code.
 - **Small storage:** a few kilobytes of source regenerates megabytes of files.
-

7. Current Status

CADIO is at **Phase 0, a working vertical slice** that runs end to end:

program → sandboxed execution → STL/STEP/GLB export → validation gate → browser viewer, with a full Claude agent loop on top.

Done and working today:

- Engine 1 (precision, build123d/OCCT) with a warm worker pool.
 - The full agent loop: ask questions, build, *see* its own renders, self-correct.
 - Projects, persistence, and conversation resume across restarts.
 - Reference image and STEP/STL geometry import.
 - Live parameter sliders, part-aware selection, parameter-to-face highlighting.
 - Version history with numbered, origin-tagged versions.
 - Google sign-in, per-user projects, and a Docker/Bifrost deployment.
 - Performance and reliability hardening (fast renders, graceful restarts, refresh-safe builds, health probes).
-

8. Future Plans and Scope

The roadmap is deliberately **one vertical at a time**: each engine reaches professional quality before the next begins. The direction is A, then B, then C.

Near term: finishing Engine 1's polish

- **Threads and knurls:** extend the corpus so the precision engine handles fastener threads and textured grips reliably.
- **Dimensioned 2D drawings:** a checkpoint artifact (an engineering drawing) before a model is called "done."
- **Resumable agent turns:** today a turn interrupted by a server restart is lost; checkpointing mid-turn would make even that survive.

Track C, Engine 2: Blender (the big breadth unlock)

Precision CAD can't do organic, sculpted, or stylized shapes; that's outside B-rep modeling by nature. The plan's answer is a **second engine: headless Blender (bpy)**, plugged into the exact same engine contract.

- Same program shape (**PARAMS** + **build()**), same worker-pool protocol, same validation and params UI, proving the pluggable-engine bet.
- The agent gains a **run_blender** tool and **routes** each request to the right engine ("precision for dimensional/functional; Blender for organic/stylized/decorative").
- Turntable renders feed the same "agent's eyes" self-critique loop.
- **Success looks like:** a twisted ribbed vase, text truly wrapped around a sphere, and a printable low-poly decorative animal, all in the same project, mixed with precision-engine versions, with the shell unchanged.

Longer term (from the product plan)

- **Engine 3:** hosted text-to-3D as *starting* meshes, then refined in Blender (using generative AI as a starting point, not a dead end).
- **Cross-engine composition:** combine parts from different engines in one model.
- **Share links:** send a project or model to someone else.
- **Print-service integration:** order a physical print straight from a model.
- **Multi-user and teams:** collaboration and scaling out (a job queue plus more compute nodes, triggered by peak concurrent builds, not user count).

Scope boundaries (what CADIO is deliberately *not*)

- Not a from-scratch sculpting tool: creation happens through conversation and parameters, not manual vertex pushing.
- Not a browser-only toy: the CAD kernel is too heavy to run on users' devices, so compute stays server-side.
- Not a slot machine: the whole design is about a collaborating agent that gathers requirements and verifies its work, not a one-shot generator.